

گزارش تمرین ۱

مهسا ابرغانی اقدام - شماره دانشجویی : ۸۱۰۱۰۳۰۵۳

الف: در این بخش سری تیلور را براساس فرمول ارائه شده در قالب یک تابع به نام `taylor` پیاده سازی می‌کنیم. بنابه خواسته‌ی روی تمرین، بدون انجام بهینه‌سازی و روی یک نود و یک هسته اجرا خواهد شد. قبل و بعد از فراخوانی تابع `taylor`، زمان سیستم را برای بدست آوردن زمان اجرا ذخیره می‌کنیم. در نهایت با استفاده از تابع `print` نتیجه تابع `taylor` و زمان اجرا چاپ می‌شود.

```
1  import time
2  import math
3
4  def taylor(n = 5000):
5      sum = 0
6      for k in range(n):
7          a = math.factorial((2*k)+1)
8          c = 2 ** ((3*k)+1)
9          d = (math.factorial(k)) ** 2
10         b = c * d
11         sum += a / b
12
13     return sum
14
15     startTime = time.time()
16     callTaylor = taylor()
17     endTime = time.time()
18
19     print( f'result = {callTaylor}' )
20     print( f'total time = {endTime - startTime}' )
21
```

خروجی :

```
aqdam@raspberrypi-dml2:~/tmrine1/1/فد $ mpirun -n 1 python sqrt_a.py
result = 1.414213562373095
total time = 18.621827363967896
aqdam@raspberrypi-dml2:~/tmrine1/1/فد $ |
```

براساس خروجی چاپ شده، مشاهده می شود خروجی تابع $taylor$ مقدار صحیح $1/41$ و زمان اجرای $18/62$ ثانیه بدست آمده است.

ب: با بررسی سری تیلور مشاهده می شود که هم در صورت و هم در مخرج کسر، بسیاری از محاسبات انجام شده با مراحل قبل همپوشانی دارند و امکان استفاده از نتایج مراحل قبل وجود دارد. به همین دلیل در این بخش به شرحی که گفته خواهد شد، تابع $taylor$ را بازنویسی و بهینه سازی می کنیم.

۱- مقداردهی اولیه: متغیرهای فاکتوریل ها و توان های مربوط به سری مقداردهی اولیه شده اند (برای $k=0$).

۲- حلقه تکرار: در هر تکرار، فاکتوریل ها و توان ها به صورت بازگشتی محاسبه می شوند تا از محاسبات تکراری جلوگیری شود.

۳- جمع آوری جملات سری: هر جمله از سری تیلور با فرمول موردنظر محاسبه شده و به مجموع نهایی اضافه می شود.

۴- بازگشت نتیجه: مقدار نهایی سری تیلور برای بازه داده شده بازگردانده می شود.

قبل و بعد از فراخوانی تابع $taylor$ ، زمان سیستم را برای بدست آوردن زمان اجرا ذخیره می کنیم. در نهایت با استفاده از تابع $print$ نتیجه تابع $taylor$ و زمان اجرا چاپ می شود.

```

> sqrt_b.py > ...
import time

startTime = time.time()
def taylor(n = 5000):
    sum = 0
    # if k == 0 :
    a_firstValue = 1
    c_firstValue = 2
    d_firstValue = 1

    for k in range(n):
        if k > 0:
            a_firstValue *= (2 * k) * ((2 * k) + 1)
            c_firstValue *= 8
            d_firstValue *= k
            sum += a_firstValue / (c_firstValue * (d_firstValue**2))

    return sum

res = taylor()
endTime = time.time()
print(f' result = {res}')
print(f'total time = {endTime - startTime}')

```

خروجی:

```

aqdam@raspberrypi-dml2:~/tmrine1/1/فد $ cd ../
aqdam@raspberrypi-dml2:~/tmrine1/1 $ cd ب
aqdam@raspberrypi-dml2:~/tmrine1/1/ب $ mpirun -n 1 python sqrt_b.py
result = 1.414213562373095
total time = 4.63758659362793
aqdam@raspberrypi-dml2:~/tmrine1/1/ب $ |

```

مشاهده می‌شود که این بار همان خروجی در حدود $\frac{4}{63}$ ثانیه اجرا می‌شود. چرا که فاکتوریل‌ها و توان‌ها به صورت بازگشتی محاسبه می‌شوند و با این روش از محاسبات تکراری جلوگیری شده است.

ج - ۱: همان الگوریتم مرحله‌ی الف را این بار برای ۱ نود و ۲ هسته اجرا می‌کنیم:

```

1 > ڄ >  sqrt_c_1.py > ...
 1  from mpi4py import MPI
 2  import time
 3  import math
 4
 5  comm = MPI.COMM_WORLD
 6  rank = comm.Get_rank()
 7  size = comm.Get_size()
 8
 9
10  n = 5000
11
12  chunk_size = n // size
13  remain = n % size
14
15  if rank < remain:
16      start = rank * (chunk_size + 1)
17      end = start + chunk_size
18  else:
19      start = rank * chunk_size + remain
20      end = start + chunk_size - 1
21
22  def taylor(start, end):
23      summation = 0
24      for k in range(start, end + 1):
25          a = math.factorial((2*k)+1)
26          c = 2 ** ((3*k)+1)
27          d = (math.factorial(k)) ** 2
28          b = c * d
29          summation += a / b
30

```

```

1 > ڄ > 🐍 sqrt_c_1.py > ...
30
31     return summation
32     startTime = time.time()
33
34     sub_result = taylor(start, end)
35
36     total_result = comm.reduce(sub_result, op=MPI.SUM, root=0)
37
38     endTime = time.time()
39
40     if rank == 0:
41         print(f"Rank {rank}: Total result with n={n}: {total_result}")
42         print('\n')
43         print(f"Rank {rank}: Total time: {endTime - startTime} seconds")
44
45
46     print('\n')
47     print(f"Rank: {rank} time: {endTime - startTime} seconds")
48

```

```

1 > ڄ > 📄 sqrt_c_1.sh
1   #!/bin/bash
2   #SBATCH --job-name=sqrt_c_1
3   #SBATCH --nodes=1
4   #SBATCH --ntasks-per-node=2
5   #SBATCH --partition=partition
6   #SBATCH --output=sqrt_c_1.out
7   echo 'jobs are started ...'
8   srun --mpi=pmix_v4 python3 sqrt_c_1.py
9

```

نتيجه:

```
aqdam@raspberrypi-dm × + ∨
GNU nano 7.2
jobs are started ...
Rank 0: Total result with n=5000: 1.414213562373095

Rank 0: Total time: 15.912672758102417 seconds

Rank: 0 time: 15.912672758102417 seconds

Rank: 1 time: 15.91263222694397 seconds
```

مشاهده می شود که همان الگوریتم روی یک نود و دو هسته در حدود ۳ ثانیه سریع تر از یک نود و یک هسته اجرا می شود. دلیل این افزایش سرعت توزیع بار محاسباتی بین دو هسته است.

ج - ۲: این بار روی دو نود و دو هسته اجرا می کنیم. بدین منظور، n که تعداد بار اجرای تابع ما می باشد، را تقسیم بر تعداد هسته می کنیم و نصفه ی اول را به یک هسته و نصفه ی دیگر را به هسته ی دیگر می دهیم تا اجرا کند. این کد به گونه ای نوشته شده است که حتی اگر تعداد هسته ها بیشتر هم شود باز هم کد به درستی کار میکند.

```
1 > ڄ >  sqrt_c_2.py > ...
 1  from mpi4py import MPI
 2  import time
 3  import math
 4
 5  comm = MPI.COMM_WORLD
 6  rank = comm.Get_rank()
 7  size = comm.Get_size()
 8
 9
10  n = 5000
11
12  chunk_size = n // size
13  remain = n % size
14
15  if rank < remain:
16      start = rank * (chunk_size + 1)
17      end = start + chunk_size
18  else:
19      start = rank * chunk_size + remain
20      end = start + chunk_size - 1
21
22  def taylor(start, end):
23      summation = 0
24      for k in range(start, end + 1):
25          a = math.factorial((2*k)+1)
26          c = 2 ** ((3*k)+1)
27          d = (math.factorial(k)) **2
28          b = c * d
29          summation += a / b
30
```

```

1 > ج > sqrt_c_2.py > ...
29         summation += a / b
30
31     return summation
32     startTime = time.time()
33
34     sub_result = taylor(start, end)
35
36     total_result = comm.reduce(sub_result, op=MPI.SUM, root=0)
37
38     endTime = time.time()
39
40     if rank == 0:
41         print(f"Rank {rank}: Total result with n={n}: {total_result}")
42         print('\n')
43         print(f"Rank {rank}: Total time: {endTime - startTime} seconds")
44
45
46     print('\n')
47     print(f"Rank: {rank} time: {endTime - startTime} seconds")
48

```

فایل .sh

```

1 > ج > sqrt_c_2.sh
1  #!/bin/bash
2  #SBATCH --job-name=sqrt_c_2
3  #SBATCH --nodes=2
4  #SBATCH --ntasks-per-node=2
5  #SBATCH --partition=partition
6  #SBATCH --output=sqrt_c_2.out
7  echo 'jobs are starter ...'
8  srun --mpi=pmix_v4 python sqrt_c_2.py
9

```

نتیجه:

```
aqdam@raspberrypi-dm × + ∨
GNU nano 7.2
jobs are started ...
Rank 0: Total result with n=5000: 1.414213562373095

Rank 0: Total time: 10.246097564697266 seconds

Rank: 0 time: 10.246097564697266 seconds

Rank: 2 time: 10.246129751205444 seconds

Rank: 1 time: 10.246106147766113 seconds

Rank: 3 time: 10.2461256980896 seconds
```

زمان اجرا بر روی دو نود و دو هسته نسبت به حالت یک نود و یک هسته و دو نود و دو هسته کاهش می‌یابد.

دلیل این افزایش سرعت توزیع بار محاسباتی بین چهار هسته است.

د-۱: این بار الگوریتم مرحله ی ب را تقسیم کرده و روی یک نود و دو هسته اجرا می‌کنیم:

1 > > >  sqrt_d_1.py > ...

```
1  | From mpi4py import MPI
2  | import time
3  | import math
4
5  | comm = MPI.COMM_WORLD
6  | rank = comm.Get_rank()
7  | size = comm.Get_size()
8
9
10 | n = 5000
11
12 | chunk_size = n // size
13 | remain = n % size
14
15 | if rank < remain:
16 |     start = rank * (chunk_size + 1)
17 |     end = start + chunk_size
18 | else:
19 |     start = rank * chunk_size + remain
20 |     end = start + chunk_size - 1
21
22 | def taylor(start, end):
23 |     sumation = 0
24 |     # if k == 0 :
25 |     a_firstValue = 1
26 |     c_firstValue = 2
27 |     d_firstValue = 1
28
29 |     for k in range(start, end + 1):
30 |         if k > 0:
```

1 > > > sqrt_d_1.py > ...

```
29     for k in range(start, end + 1):
30         if k>0:
31             a_firstValue *= (2 * k) * ((2 * k) + 1)
32             c_firstValue *= 8
33             d_firstValue  *= k
34             summation += a_firstValue / (c_firstValue * (d_firstValue**2))
35
36     return summation
37
38     startTime = time.time()
39     sub_result = taylor(start, end)
40     total_result = comm.reduce(sub_result, op=MPI.SUM, root=0)
41     endTime = time.time()
42
43     if rank == 0:
44         print(f"Rank {rank}: Total result with n={n}: {total_result}")
45         print('\n')
46         print(f"Rank {rank}: Total time: {endTime - startTime} seconds")
47
48
49     print('\n')
50     print(f"Rank: {rank} time: {endTime - startTime} seconds")
51
```

فایل sh:.

1 > > > sqrt_d_1.sh

```
1  #!/bin/bash
2  #SBATCH --job-name=sqrt_d_1
3  #SBATCH --nodes=1
4  #SBATCH --ntasks-per-node=2
5  #SBATCH --partition=partition
6  #SBATCH --output=sqrt_d_1.out
7  echo 'jobs are started ...'
8  srun --mpi=pmix_v4 python3 sqrt_d_1.py
9
```

نتیجه:

```
aqdam@raspberrypi-dm × + v
GNU nano 7.2                                sqrt_d_1.out
jobs are started ...
Rank 0: Total result with n=5000: 1.414413442476956

Rank 0: Total time: 1.0444746017456055 seconds

Rank: 0 time: 1.0444746017456055 seconds

Rank: 1 time: 1.044450044631958 seconds
```

روی یک نود و دو هسته سرعت بالاتر و زمان کمتر است، نسبت به یک نود و یک هسته و دلیل این نتیجه توزیع بار محاسباتی بر روی چند هسته می باشد.

۵-۲:

1 > > >  sqrt_d_2.py > ...

```
1  from mpi4py import MPI
2  import time
3  import math
4
5  comm = MPI.COMM_WORLD
6  rank = comm.Get_rank()
7  size = comm.Get_size()
8
9
10 n = 5000
11
12 chunk_size = n // size
13 remain = n % size
14
15 if rank < remain:
16     start = rank * (chunk_size + 1)
17     end = start + chunk_size
18 else:
19     start = rank * chunk_size + remain
20     end = start + chunk_size - 1
21
22 def taylor(start, end):
23     summation = 0
24     # if k == 0 :
25     a_firstValue = 1
26     c_firstValue = 2
27     d_firstValue = 1
28
29     for k in range(start, end + 1):
```

1 > > > sqrt_d_2.py > ...

```
29     for k in range(start, end + 1):
30         if k>0:
31             a_firstValue *= (2 * k) * ((2 * k) + 1)
32             c_firstValue *= 8
33             d_firstValue *= k
34             summation += a_firstValue / (c_firstValue * (d_firstValue**2))
35
36     return summation
37
38     startTime = time.time()
39     sub_result = taylor(start, end)
40     total_result = comm.reduce(sub_result, op=MPI.SUM, root=0)
41     endTime = time.time()
42
43     if rank == 0:
44         print(f"Rank {rank}: Total result with n={n}: {total_result}")
45         print('\n')
46         print(f"Rank {rank}: Total time: {endTime - startTime} seconds")
47
48
49     print('\n')
50     print(f"Rank: {rank} time: {endTime - startTime} seconds")
51
```

فایل sh:.

```
1 > > > sqrt_d_2.sh
1  #!/bin/bash
2  #SBATCH --job-name=sqrt_d_2
3  #SBATCH --nodes=2
4  #SBATCH --ntasks-per-node=2
5  #SBATCH --partition=partition
6  #SBATCH --output=sqrt_d_2.out
7  echo 'jobs are starter ...'
8  srun --mpi=pmix_v4 python sqrt_d_2.py
9
```

نتیجه:

```
aqdam@raspberrypi-dm x + v
GNU nano 7.2 sqrt_d_1.out
jobs are started ...
Rank 0: Total result with n=5000: 1.4149462433375248

Rank 0: Total time: 0.23613619804382324 seconds

Rank: 0 time: 0.23613619804382324 seconds

Rank: 3 time: 0.2356860637664795 seconds

Rank: 1 time: 0.23610901832580566 seconds

Rank: 2 time: 0.23570895195007324 seconds
```

روی دو نود و دو هسته نیز زمان اجرا نسبت به یک نود و دو هسته کمتر است و دلیل این نتیجه توزیع بار محاسباتی بر روی چند هسته می باشد.

ه: این بار باید طوری عمل کنیم که زمان کاهش یابد. به دلیل اینکه در فاکتوریل هرچه عدد بزرگ تر شود، تعداد ضرب ها بیشتر میشود. همین موضوع باعث می شود که هسته ی اول که ضرب های تا ۲۵۰۰ را برعهده دارد، زمانی که کارش تمام می شود. هنوز هسته ی دوم که ضرب های ۲۵۰۱ تا ۵۰۰۰ را بر عهده دارد تمام نشده (به دلیل تعداد ضرب بالا). به همین دلیل باعث می شود توزیع به صورت نامتوازن انجام شود.

در این مرحله به جای اینکه مثلاً ۱ تا ۵ را به اولی، ۶ تا ۱۰ را به دومی، ۱۱ تا ۱۵ را مجدد به اولی، ۱۶ تا ۲۰ را مجدد به دومی و الی آخر تخصیص دهیم، جوری تنظیم می کنیم که ۱ برای اولی، ۲ برای دومی، ۳ برای سومی، ۴ برای چهارمی، مجدداً ۵ برای اولی، ۶ برای دومی و ... باشد و اینگونه می شود که سرعت محاسبات و پردازش بالا می رود و در زمان کمتری اجرا می شود.

1 > 0 >  sqrt_e.py > ...

```
1  from mpi4py import MPI
2  import time
3  import math
4
5  comm = MPI.COMM_WORLD
6  rank = comm.Get_rank()
7  size = comm.Get_size()
8
9
10 n = 5000
11
12
13 def taylor(rank, size, n):
14     summation = 0
15     for k in range(rank, n + 1, size):
16         a = math.factorial((2*k)+1)
17         c = 2 ** ((3*k)+1)
18         d = (math.factorial(k)) **2
19         b = c * d
20         summation += a / b
21
22     return summation
23
24 startTime = time.time()
25 sub_result = taylor(rank, size, n)
26 total_result = comm.reduce(sub_result, op=MPI.SUM, root=0)
27 endTime = time.time()
28
29 if rank == 0:
30     print(f"Rank {rank}: Total result with n={n}: {total_result}")
```

1 > 0 >  sqrt_e.py > ...

```
29 if rank == 0:
30     print(f"Rank {rank}: Total result with n={n}: {total_result}")
31     print('\n')
32     print(f"Rank {rank}: Total time: {endTime - startTime} seconds")
33
34
35 print('\n')
36 print(f"Rank: {rank} time: {endTime - startTime} seconds")
37
```

فایل `sh`:

```
1 > ۵ >  sqrt_e.sh
1   #!/bin/bash
2  #SBATCH --job-name=sqrt_e
3  #SBATCH --nodes=2
4  #SBATCH --ntasks-per-node=2
5  #SBATCH --partition=partition
6  #SBATCH --output=sqrt_e.out
7  echo 'jobs are starter ...'
8  srun --mpi=pmix_v4 python sqrt_e.py
9
```

نتیجه:

```
aqdam@raspberrypi-dm × + ▾
GNU nano 7.2 sqrt_e.out
jobs are started ...
Rank 0: Total result with n=5000: 1.414213562373095

Rank 0: Total time: 4.766910791397095 seconds

Rank: 0 time: 4.766910791397095 seconds

Rank: 1 time: 4.766943693161011 seconds

Rank: 2 time: 4.767081260681152 seconds

Rank: 3 time: 4.767068386077881 seconds
```